부산대학교
PUSAN NATIONAL UNIVERSITY

Linked-List Operations

Walking the References

Traversal, Search, Insertion, and Removal

DS3600082-140
자료구조
2026학년도 2학기

오용경
데이터사이언스전공
yongkyung.oh@pusan.ac.kr

Traversal

`head`나 `tail`이 가리키지 않는 노드를 찾으려면 처음부터 연결을 따라간다

끝과 가운데의 접근 차이

머리(head)와 꼬리(tail)가 직접 가리키지 않는 가운데 노드에는 순회(traversal)로 접근한다

건드리는 자리	접근 비용	접근 방법
앞쪽 끝	$\Theta(1)$	`head`가 바로 가리킴
뒤쪽 끝	$\Theta(1)$	`tail`이 바로 가리킴
가운데	$\Theta(1+i)$	`head`부터 `next`를 따라감

> 머리나 꼬리가 직접 가리키는 위치에는 $\Theta(1)$에 접근할 수 있다

Source: Morin (2013), Section 3.1.

순회를 코드로 옮기면

현재 노드를 가리키는 `current`를 한 칸씩 옮긴다

```
def size(head):
    current = head
    count = 0
    while current is not None:
        count += 1
        current = current.next
    return count

```

> 반복 한 번이 노드 하나를 방문하므로 전체 비용은 $\Theta(n)$이다

Adapted from Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

`size`가 네 노드를 세는 과정

반복문은 현재 노드를 세고, 다음 참조를 읽은 뒤 다시 조건을 검사한다

반복 전	`current`	반복 뒤 `count`
처음	54	1
두 번째	26	2
세 번째	93	3
네 번째	17	4
종료 검사	`None`	그대로 4

> `None`은 노드가 아니므로 세지 않고, 반복문 종료를 알린다

순회

정의

순회(traversal)

`head`에서 시작해 `next`를 차례로 따라가며 노드를 하나씩 방문하는 일. `None`을 만나면 끝난다

Source: Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

빈 목록에서는 반복문을 건너뛴다

head가 None이면 첫 조건 검사에서 곧바로 멈춘다

```
empty_head = None
print(size(empty_head))  # 0

one = Node(54)
print(size(one))         # 1

```

순회 코드는 빈 목록과 원소가 하나인 목록을 별도 분기 없이 처리한다

i번째 노드를 찾는 반복

head에서 시작하므로 목표 인덱스만큼 next를 따라간다

```
def node_at(head, i):
    current = head
    for _ in range(i):
        current = current.next
    return current

# Assume 0 <= i < n.

```

$i = 0$이면 이동하지 않고, $i = n - 1$이면 $n - 1$번 참조를 따라간다

`node_at(head, 2)`의 이동 추적

54 → 26 → 93 → 17에서 인덱스 2의 노드를 찾는다

단계	`current`	따라간 횟수
시작	54 (`head`)	0
반복 1	26	1
반복 2	93	2
반환	93	$i = 2$번 이동으로 끝남

*i*번 따라가므로 *i*번째 노드에 이르는 비용은 $\Theta(1 + i)$이고, $i = n - 1$이면 $\Theta(n)$이다

Adapted from Morin (2013), Section 3.1.

배열과 연결리스트의 순회 종료 조건

배열은 인덱스를 크기와 비교하고 연결리스트는 `None` 참조를 확인한다

표현	종료 조건	확인 방법
배열	인덱스가 n에 도달	경계를 수로 비교
연결리스트	`current`가 `None`	끝을 나타내는 참조 확인

> 끝을 알리는 방식이 다르므로 반복문의 조건도 다르다

Source: Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

순회를 중첩하면 비용이 달라진다

같은 순회를 연속하면 비용이 상수 배로 늘지만, 순회 안에서 다시 순회하면 n^2에 비례한다

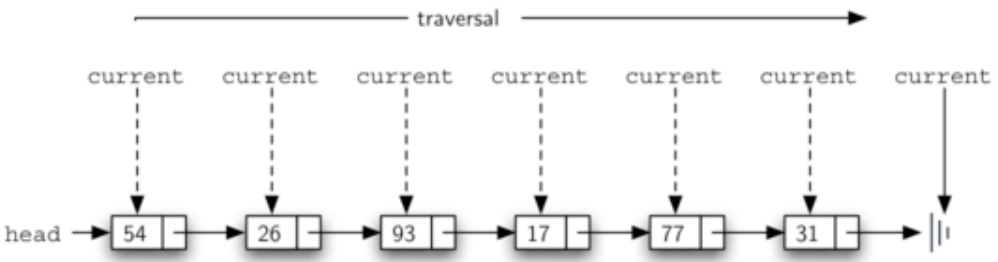
하려는 일	순회 횟수	비용
값을 모두 더하기	1	$\Theta(n)$
최댓값과 최솟값을 따로	2	$\Theta(n)$
각 원소마다 목록을 다시 훑음	n	$\Theta(n^2)$

> 상수 배는 복잡도 등급을 바꾸지 않지만 순회 안에 순회를 넣으면 바뀐다

Adapted from Morin (2013), Section 1.3.3.

배열과 연결리스트의 전체 순회

모든 원소를 한 번씩 훑는 비용은 두 구조 모두 $\Theta(n)$이다



한 번의 순회에서 `current`가 거치는 노드를 모두 표시했다.

Source: Miller and Ranum (2013), Linked Lists.

순회를 멈추는 조건

멈추는 조건이 방문하는 노드 수를 정한다

멈추는 조건	언제 멈추나	비용
끝까지	`None`을 만남	늘 $\Theta(n)$
찾으면	조건을 만족하는 노드	최대 $\Theta(n)$
i번 지나면	센 값이 i가 됨	$\Theta(1 + i)$

> 같은 순회라도 멈추는 조건에 따라 비용이 다르다

Source: Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

Checkpoint

Traversal

- 순회 한 걸음이 하는 일은 무엇인가
- i번째 자리에 가는 비용이 $\Theta(1 + i)$인 까닭은 무엇인가
- `current`가 `None`일 때 무엇을 알 수 있는가

Section 2 • Linked-List Operations

Search

값을 만나거나 끝에 이를 때까지 순회한다

탐색

정의

탐색(search)

찾는 값이 목록에 있는지 확인하는 일. 순회하며 각 노드의 값을 비교해 찾으면 멈추고, `None`에 이르면 값이 없다고 판단한다

Source: Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

탐색의 세 경우

최선의 경우·최악의 경우·평균 경우를 탐색에 적용한다

경우	해당 입력	비교 횟수
최선의 경우	첫 노드가 답	1
최악의 경우	마지막이거나 없음	n
평균 경우	찾는 값의 위치가 고르게 분포	약 $n/2$

Adapted from Miller and Ranum (2013), The Sequential Search, Table 1.

탐색의 멈춤 조건

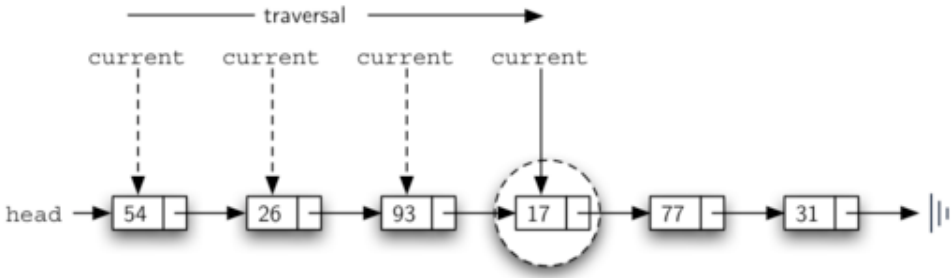
값을 찾으면 즉시 반환하고, None에 이르면 실패한다

```
def search(head, target):
    current = head
    while current is not None:
        if current.data == target:
            return True
        current = current.next
    return False

```

Adapted from Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

current가 지나가는 자리



값 17을 찾을 때 current는 앞에서부터 네 노드를 차례로 방문한다.

Source: Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

찾는 위치에 따라 비교 횟수가 바뀐다

54 → 26 → 93 → 17에서 같은 코드를 여러 목표에 적용한다

목표	비교한 값	결과
54	54	1번 뒤 성공
93	54, 26, 93	3번 뒤 성공
17	54, 26, 93, 17	4번 뒤 성공
42	54, 26, 93, 17	4번 뒤 실패

실패한 탐색에서는 마지막 노드까지 검사한 뒤 `None`을 만나야 한다

탐색 실패에 필요한 비교

찾는 값이 없으면 모든 노드를 확인해야 한다

경우	멈추는 때	비교 횟수
값이 있음	값을 찾은 순간	평균 약 $n/2$
값이 없음	`None`에 도달	항상 n

> 정렬되지 않은 목록에서 값이 없음을 확인하려면 모든 원소를 검사해야 한다

Source: Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

Checkpoint

Search

- 찾는 값이 없을 때 모든 노드를 확인해야 하는 까닭은 무엇인가
- 첫 노드에서 값을 찾으면 비교는 몇 번 하는가
- 탐색 코드가 `False`를 반환하는 조건은 무엇인가

Insert

뒤쪽 참조를 보존한 뒤 앞쪽 참조를 바꾼다

삽입

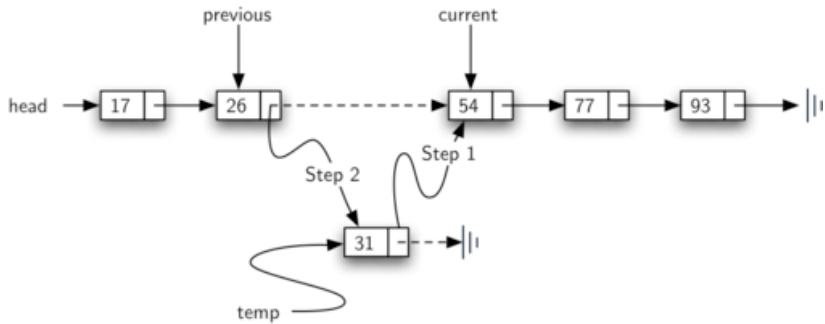
삽입(insert)의 비용은 자리를 찾는 일과 연결을 고치는 일로 나누어 센다

단계	비용	하는 일
자리 찾기	$\Theta(1+i)$	순회
연결 고치기	$\Theta(1)$	참조 두 개

Source: Morin (2013), Section 3.1.

연결을 고치는 순서

`previous`는 넣을 자리의 앞 노드, `current`는 뒤 노드를 가리키는 순회 변수다



1단계에서 새 노드 31이 `current`(54)를 가리키게 하고, 2단계에서 `previous`(26)의 `next`가 새 노드를 가리키게 한다.

Source: Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

가운데에 넣을 때 보존할 경로

previous가 26, current가 54를 가리킬 때 31을 사이에 둔다

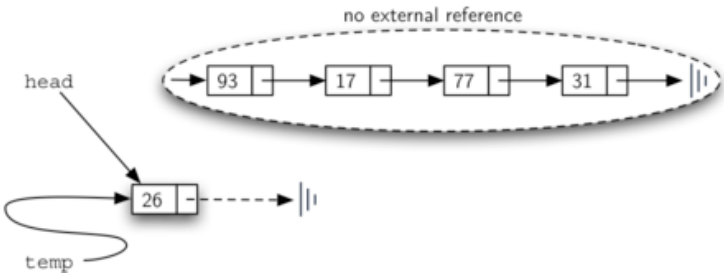
```
previous = node_at(items.head, i - 1)   # 1 <= i < n
current = previous.next
new_node = Node(31)
new_node.next = current
previous.next = new_node
items.n += 1

```

$26 \to 54$였던 경로가 $26 \to 31 \to 54$가 된다.

> node_at으로 자리를 찾는 데 $\Theta(1 + i)$, 연결을 고치는 데 $\Theta(1)$이 든다

맨 앞 삽입에서 순서를 바꾸면 뒤쪽을 잃는다



`head`를 먼저 새 노드로 바꾸면 기존 첫 노드를 가리키는 참조가 하나도 남지 않는다.

Caution 새 노드의 `next`를 먼저 연결한 뒤 `head`를 옮긴다

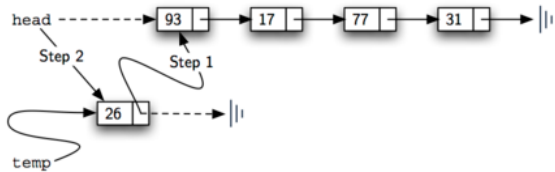
Source: Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

맨 앞에 넣을 때 두 참조를 바꾸는 순서

93 → 17 → 77 → 31 앞에 26을 넣을 때 새 노드에서 기존 첫 노드를 먼저 가리킨다

```
new_node = Node(26)
new_node.next = items.head
items.head = new_node
if items.tail is None:
    items.tail = new_node
items.n += 1

```



그림의 `temp`는 코드의 `new_node`다. Step 1이 둘째 줄, Step 2가 셋째 줄이다.

> 빈 목록에서는 `head`와 `tail`이 모두 새 노드를 가리킨다

Adapted from Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

Checkpoint

Insert

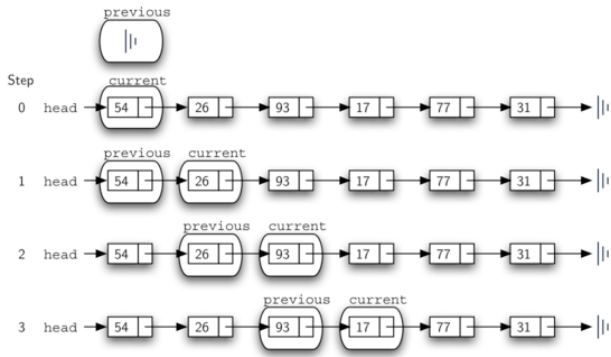
- 맨 앞 삽입에서 참조를 고치는 순서가 중요한 까닭은 무엇인가
- 맨 앞에 새 노드를 넣을 때 먼저 연결해야 하는 참조는 무엇인가
- *i*번째 자리를 찾는 비용과 연결을 고치는 비용은 각각 얼마인가

Section 4 • Linked-List Operations

Removal and Its Cost

앞 노드와 현재 노드를 함께 추적해 지울 노드를 건너뛰도록 잇는다

previous와 current를 함께 옮긴다



previous는 현재 노드의 앞을, current는 검사할 노드를 가리킨다.

Source: Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

삭제할 노드를 찾는 반복

previous와 current를 함께 옮기며 지울 값을 찾는다

```
previous = None
current = items.head
while current is not None and current.data != item:
    previous = current
    current = current.next
if current is None:
    raise ValueError(item)    # empty list or item absent

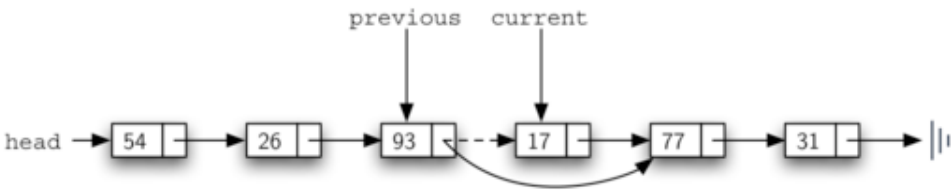
```

> current가 None이면 빈 목록이거나 값이 없고, previous가 None이면 첫 노드를 지운다

Adapted from Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

삭제

삭제(remove)는 지울 노드의 앞 노드가 그 다음을 가리키게 한다



`previous.next`가 `current.next`를 가리키면 노드 17은 목록에서 더 이상 도달할 수 없다

> 원소를 옮기지 않고 참조 하나만 바꾼다

Source: Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

첫 노드를 지울 때



경우	`previous`	고칠 것
가운데 노드	앞 노드가 있음	`previous.next`
첫 노드	없음	`head`

Caution `previous`가 `None`인 경우를 따로 처리하지 않으면 첫 노드를 삭제할 수 없다

Source: Miller and Ranum (2013), Implementing an Unordered List: Linked Lists.

첫 노드를 지우면 head를 옮긴다

앞 노드가 없으므로 previous.next 대신 head를 바꾼다

```
removed = items.head        # Assume nonempty.
items.head = removed.next
items.n -= 1
if items.head is None:
    items.tail = None

```

원소가 하나였다면 head와 tail이 함께 None이 된다

가운데나 끝의 노드를 지우면

앞 노드가 지울 노드의 다음을 가리키게 하고, 끝이었다면 `tail`도 옮긴다

```
# Assume previous and current are known.
previous.next = current.next
if current is items.tail:
    items.tail = previous
items.n -= 1

```

연결 변경 자체는 일정한 시간이지만, `previous`를 찾는 순회는 최대 $\Theta(n)$이다

연산별 비용

연산	비용	비용을 결정하는 요인
순회	$\Theta(n)$	노드 수
탐색	최악 $\Theta(n)$	비교 횟수
앞에 삽입	$\Theta(1)$	참조 두 개
i에 삽입	$\Theta(1+i)$	자리를 찾는 순회
i에서 삭제	$\Theta(1+i)$	자리를 찾는 순회
끝에 삽입	$\Theta(1)$	`tail.next`와 `tail`
끝에서 삭제	$\Theta(n)$	`tail` 앞까지 $n-2$번 이동

Source: Morin (2013), Sections 3.1 and 3.1.2.

Checkpoint

Removal and Its Cost

- `previous`와 `current`는 각각 어느 노드를 가리키는가
- 첫 노드를 지울 때 따로 처리해야 하는 것은 무엇인가
- *i*번째 노드를 지우는 비용을 결정하는 단계는 무엇인가

Which Cost Dominates, Finding or Fixing?

Next Lecture: Doubly Linked Lists

- 양방향 연결과 센티널
- 끝에서 삭제를 $\Theta(1)$로 만드는 방법
- 노드를 이미 알고 있을 때의 삽입·삭제 비용

오용경 yongkyung.oh@pusan.ac.kr